
Python



Author - Vishal Kumar

Overview

- INTRODUCTIONS
 - VARIABLES
 - DATA TYPES
 - OPERATORS
 - CONDITIONAL STATEMENT
 - LOOPS
 - FUNCTIONS, MODULES AND PACKAGES
 - FILE HANDLING
 - EXCEPTION HANDLING
 - OOPs CONCEPT
 - REGEX EXPRESSION
-

Introductions

- Introduction
- Applications
- Setup and Installations
- IDEs

Introdunction

- Easy to learn & easy to read
 - Open source, high level, interpreted and dynamically typed
 - Large standard library
 - Supports both functional programming and Object Oriented programming
 - It can be easily integrated with C, C++, Java etc
-

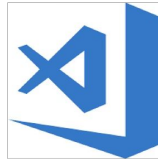
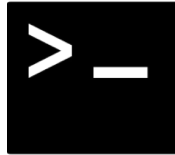
Applications

- Web development
 - Scripting
 - Game development
 - Scientific and Numeric applications
 - Artificial Intelligence and Machine Learning
 - Web scraping
 - Image processing and Graphic design applications
-

Setup and Installations

- Go to <https://www.python.org/downloads/>
 - Download the python version you need
 - Double click and then install it.
-

IDEs



Variables

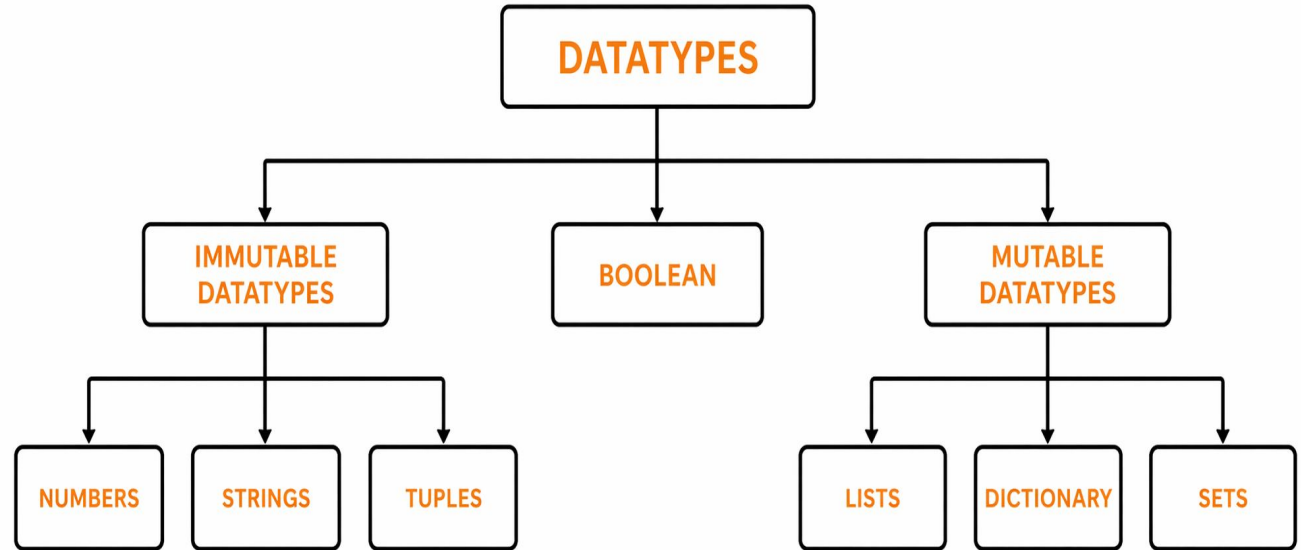
Variables

- Variables are containers for storing data values. Ex. `y = "John"`, `x = 5`
 - Python is dynamically typed
 - Variables can start with a letter or the underscore character
 - Variables can only contain alpha-numeric characters and underscores (A-z, 0-9, and `_`)
 - Variables are case-sensitive (`age`, `Age` and `AGE` are three different variables)
-

Data Types

- Types of data types
 - Boolean
 - Numbers
 - String
 - List
 - Tuples
 - Sets
 - Dictionary
-

Data Types



Boolean

- True or False
 - What all are True in Python?
 - Any number is True, except 0
 - Any string is True, except empty strings
 - Any list, tuple, set or dictionary is True, except empty ones
 - So what's left in False?
 - `()`, `{}`, `[]`, `""`, `0`, `None`
-

Numbers

- int, float and complex (1, 2, 5.6 , 2+3j)
 - Int, or integer, is a whole number, positive or negative, without decimals, of unlimited length.
 - Ex: 1, 34383475, -34
 - Float, or "floating point number" is a number, positive or negative, containing one or more decimals.
 - Ex: 35e3, 1.4, 5.0
 - Complex numbers are written with a "j" as the imaginary part.
 - Ex: 2+3j, -4j, 6j
-

Strings

- Strings can be denoted by single quotes (') or double quotes ("). Multiline strings are denoted using triple-quotes ("""...""") or (''...'')
 - Ex: "Hello", 'Python', """ Multiline strings are denoted using triple quotes like this"""
-

find()	Searches the string for a specified value and returns the position of where it was found
format()	Formats specified values in a string
isalnum()	Returns True if all characters in the string are alphanumeric
join()	Joins the elements of an iterable to the end of the string
lower()	Converts a string into lower case
replace()	Returns a string where a specified value is replaced with a specified value
split()	Splits the string at the specified separator, and returns a list
startswith()	Returns true if the string starts with the specified value
swapcase()	Swaps cases, lower case becomes upper case and vice versa
upper()	Converts a string into upper case

Lists

- Ordered, mutable collection of objects of same type or different type.
 - Creating a list:
 - `myList = [1, 'a', [1,2,3], "Hello", 789]`
 - `myList = list( )` OR `myList = [ ]`
 - Slicing: Get any part of the list by slicing. It returns a new list
 - Ex. `myList[1:3]`, `myList[1:-1]`, `myList[:3]`, `myList[3:]`, `myList[:]`
 - Indexing: Get any item in the list. It returns an item.
 - Ex. `myList[0]`, `myList[-1]`
-

append()	Adds an element at the end of the list
extend()	Add the elements of a list (or any iterable), to the end of the current list
insert()	Adds an element at the specified position
remove()	Removes the item with the specified value
del	Deletes the element
pop()	Removes the element at the specified position
clear()	Removes all the elements from the list
copy()	Returns a copy of the list
reverse()	Reverses the order of the list
sort()	Sorts the list
count()	Returns the number of elements with the specified value
index()	Returns the index of the first element with the specified value

Tuples

- Ordered, immutable collection of objects of same type or different type.
 - Creating a tuple:
 - `myTuple = ("a", "b", "mpilgrim", "z", "example")`
 - `myTuple = ()` or `tuple()`
 - Slicing: Get any part of the list by slicing. It returns a new list
 - Ex. `myTuple[1:3]`, `myTuple[1:-1]`, `myTuple[:3]`, `myTuple[3:]`, `myTuple[:]`
 - Indexing: Get any item *i* in the list. It returns an item.
 - Ex. `myTuple[0]`, `myTuple[-1]`
 - Note: `append()`, `extend()`, `insert()`, `remove()`, and `pop()` doesn't work
-

count()	Returns the number of times a specified value occurs in a tuple
index()	Searches the tuple for a specified value and returns the position of where it was found

Sets

- Collection which is unordered and unindexed.
 - Creating a set:
 - `mySet = {"apple", "banana", "cherry"}` or `mySet = set()`
-

add()	Adds an element to the set
symmetric_difference()	Returns a set with the symmetric differences of two sets
union()	Return a set containing the union of sets
difference()	Returns a set containing the difference between two or more sets
difference_update()	Removes the items in this set that are also included in another, specified set
discard()	Remove the specified item
intersection()	Returns a set, that is the intersection of two other sets
intersection_update()	Removes the items in this set that are not present in other, specified set(s)
isdisjoint()	Returns whether two sets have a intersection or not
issubset()	Returns whether another set contains this set or not
issuperset()	Returns whether this set contains another set or not
pop()	Removes an element from the set

Dictionary

- Unordered set of key-value pairs
 - Creating a dictionary:
 - `myDict = {'name': 'Anusha', 'country': 'India'}` OR `myDict = dict( )`
 - No duplicate keys
 - Add new key-value pair at any time
 - Values can be of any datatype but keys must be immutable datatypes.
 - Access values using keys Ex. `myDict['name']`
-

clear()	Removes all the elements from the dictionary
copy()	Returns a copy of the dictionary
fromkeys()	Returns a dictionary with the specified keys and value
get()	Returns the value of the specified key
items()	Returns a list containing a tuple for each key value pair
keys()	Returns a list containing the dictionary's keys
pop()	Removes the element with the specified key
popitem()	Removes the last inserted key-value pair
setdefault()	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
update()	Updates the dictionary with the specified key-value pairs
values()	Returns a list of all the values in the dictionary

Operators

Operators

- Arithmetic Operators + - * / % ** //
 - Comparison Operators == != > < >= <=
 - Assignment Operators =
 - Logical Operators. and or not
 - Identity Operators is is not
 - Membership Operators in not in
 - Bitwise Operators & | ^ ~ << >>
-

Conditional Statement

- If
- If-else
- elif

Conditional Statement

- If Statement

```
if:  
a = 33  
b = 200  
if b > a:  
    print("b is greater than a")
```

- Elif Statement

```
if b > a:  
    print("b is greater than a")  
elif a == b:  
    print("a and b are equal")  
else:  
    print("a is greater than b")
```

- If else:

```
if b > a:  
    print("b is greater than a")  
else:  
    print("b is not greater than a")
```

Loops

- While
- For
- Break and continue

Conditional Statement

- For loop

```
for x in range(6):  
    print(x)
```

Iterate over list, tuples,
dictionary, string

- While loop

```
while i < 6:  
    print(i)  
    i += 1
```

- **break** — to exit loop when some condition is met
 - **continue** — stop current iteration and continue with the next
-

Functions

Functions

Function is a block of code which runs when called.

```
def myFunc( ):
    print("This is python function")
```

Calling:

```
myFunc( )
```

Arguments:

- Default arguments
 - Positional arguments
 - Keyword arguments
 - Variable length arguments
-

File Handling

File Handling

```
f = open('file_name', mode)
```

Or

```
with open('file_name', 'r') as fp:  
    data = fp.read( )
```

Modes of opening file:

- r - read
- w - write
- a - append
- x - create

In addition to above, t or b can be mentioned to handle file in binary or text mode

Exception Handling

Exception Handling

- Syntax errors, also known as parsing errors, are perhaps the most common kind of complaint you get while you are still learning Python.
 - Even if a statement or expression is syntactically correct, it may cause an error when an attempt is made to execute it. Errors detected during execution are called exceptions.
 - Ex. `10/0` — Zero division error
 - Try-except-finally
-

Object-Oriented Programming System
